Type Inference and Type Annotation in TypeScript

Type Annotation

Definition: Type annotation is when *you* explicitly tell TypeScript the type of a variable, parameter, or return value using the `: type` syntax.

```
let age: number = 25;
let name: string = "Riya";
let isActive: boolean = true;
let scores: number[] = [90, 85, 70];

function add(a: number, b: number): number {
  return a + b;
}
```

Here, you are the one declaring the types. TypeScript just checks that you follow them.

```
age = "twenty"; // Error: Type 'string' is not assignable to type 'number'
```

Type Inference

Definition: Type inference is when *TypeScript automatically figures out* the type of a variable based on the value assigned to it, without you writing the type.

```
let age = 25;          // inferred as number
let name = "Riya";     // inferred as string
let isActive = true;   // inferred as boolean

age = "twenty"; // Error, because TypeScript already inferred 'number'
```

Inference also works for function return types and in other situations:

```
function multiply(a: number, b: number) {
  return a * b; // return type inferred as number
}

const nums = [1, 2, 3];          // inferred as number[]
const user = { id: 1, name: "A" }; // inferred as { id: number; name: string }
```

Contextual typing is a special kind of inference where the type comes from the surrounding context:

```
const names = ["Amit", "Sara"];
names.forEach(n => console.log(n.toUpperCase())); // 'n' is inferred as string
```

Quick Comparison

Feature	Type Annotation	Type Inference
Who decides the type?	The developer	The compiler
Syntax	`let x: number = 5`	`let x = 5`
Best used for	Function parameters, public APIs, empty variables	Local variables with initial values
Verbosity	More code	Less code

When Annotation Is Required

Inference cannot work if there is no value to look at, so annotate in these cases:

```
// 1. Declared without initial value (becomes 'any' otherwise)
let result: number;
result = 10;

// 2. Function parameters (TypeScript can't guess them)
function greet(name: string) { ... }

// 3. When you want a wider type than the value suggests
let id: number | string = 5;
```

const vs let Inference

```
let a = "hello";    // type: string (can change)
const b = "hello";  // type: "hello" (literal type, can never change)
```

With `const`, TypeScript infers the narrowest possible type, called a *literal type*.

Best Practice

Let TypeScript infer where it is obvious, and annotate where it adds clarity or safety, mainly function parameters, return types of exported functions, and variables without initial values.

Interview Questions

1. What is the difference between type annotation and type inference?

Annotation is manually specifying a type (`let x: number = 5`). Inference is the compiler automatically deducing the type from the assigned value (`let x = 5`).

2. What type does TypeScript infer here? `let x;`

`any`. With no initial value, TypeScript can't infer anything, so it falls back to `any`. This loses type safety, so annotate it: `let x: number;`.

3. What is the difference between these two?

```
let a = "hi";
const b = "hi";
```

`a` is inferred as `string`, `b` as the literal type `"hi"`, because a `const` can't be reassigned.

4. Can TypeScript infer function parameter types?

Generally no, you must annotate them. The exception is contextual typing, such as callbacks passed to typed functions like `map` or `forEach`.

5. What will be the inferred return type?

```
function check(n: number) {
  if (n > 0) return "positive";
  return 0;
}
```

`"positive" | 0`, which is a union of the possible return values.

6. Should you always annotate everything? Why or why not?

No. Over-annotating adds noise. Best practice is to rely on inference for simple local variables and annotate function parameters, public API return types, and uninitialized variables.

7. What is contextual typing?

When TypeScript infers a type from where an expression is used, rather than from the value itself. Example: in `window.onclick = e => ...`, `e` is inferred as `MouseEvent`.

8. What is "best common type" inference?

When inferring an array like `[1, "a", true]`, TypeScript picks a union of the element types: `(string | number | boolean)[]`.

9. What is the output of this code, and why?

```
let value = 10;
value = "ten";
```

Compile-time error. `value` was inferred as `number`, so assigning a string is not allowed.

10. Why might you annotate a function's return type even though TypeScript can infer it?

It documents the intended contract, catches mistakes inside the function early (if you return the wrong type, the error appears at the return), and speeds up compilation on large codebases.

One-Line Summary

> **Annotation** = *you tell* TypeScript the type.
>
> **Inference** = *TypeScript figures out* the type itself.